Backend Schema Plan

TigerWatch Platform

Automated Camera Trap Triage & Individual Tiger Movement Intelligence System

Pench Tiger Reserve

Document Version: 1.0

Date: August 16, 2026

Type: Backend Schema & Architecture Plan

Tech Stack: Python 3.11+ · FastAPI · PostgreSQL 16 · PostGIS 3.4 · pgvector · Redis · Celery · MinIO

Companion Docs: PRD v1.0 · TRD v1.0 · UI/UX Design System v1.0 · Website Flow Guide v1.0

CONFIDENTIAL — FOR AUTHORIZED PERSONNEL ONLY

Table of Contents

1. Executive Summary

This document defines the complete backend schema plan for the **TigerWatch Platform** — an end-to-end AI/ML system designed for the Pench Tiger Reserve. The platform automates camera trap image triage, individual tiger identification via stripe-pattern biometrics, spatial occupancy analysis, and behavioural deviation alerting.

12	45	4	5
DATABASE TABLES	API ENDPOINTS	AI/ML MODULES	RBAC ROLES

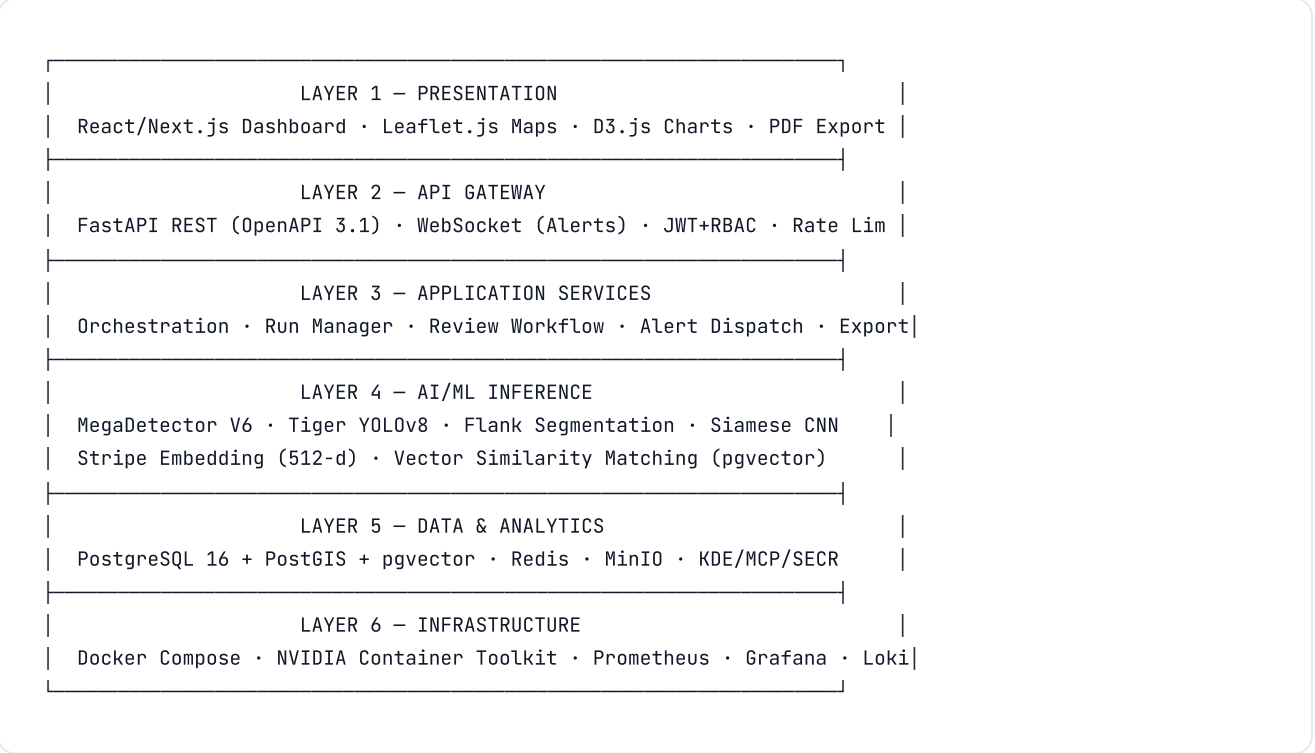
Key Backend Capabilities

- **Module 1 — Blank Image Filtering:** MegaDetector V6 (YOLOv9-C) filters 70-95% blank images with >95% recall.
- **Module 2 — Tiger Identification:** Siamese CNN with DenseNet-169 backbone extracts 512-d stripe embeddings for individual re-identification (Rank-1 accuracy >90%).
- **Module 3 — Spatial Occupancy:** KDE/MCP home-range estimation with PostGIS spatial analytics, pairwise territorial overlap analysis.
- **Module 4 — Deviation Alerting:** Statistical comparison against per-individual historical baselines; generates classified alerts for range shifts, novel stations, buffer-zone approaches, and prolonged absences.

> 📋 **Scope**
>
> This plan covers the database schema, API layer, processing pipeline, security, infrastructure, and deployment topology. Frontend implementation is covered in the companion UI/UX Design System and Website Flow documents.

2. System Architecture Overview

The system follows a **six-layer pipeline architecture** with a hybrid Edge-to-Cloud deployment model optimized for the remote, low-connectivity environment of a tiger reserve.



Inter-Component Communication

PATTERN	TECHNOLOGY	USE CASE
Synchronous	REST API (30s timeout)	Dashboard queries, image retrieval, alert management
Asynchronous	Celery + Redis broker	All AI/ML inference, batch processing pipeline
Event-Driven	PostgreSQL LISTEN/NOTIFY	Real-time alert propagation to WebSocket
File-Based	MinIO (S3-compatible)	Image persistence, thumbnails, crops

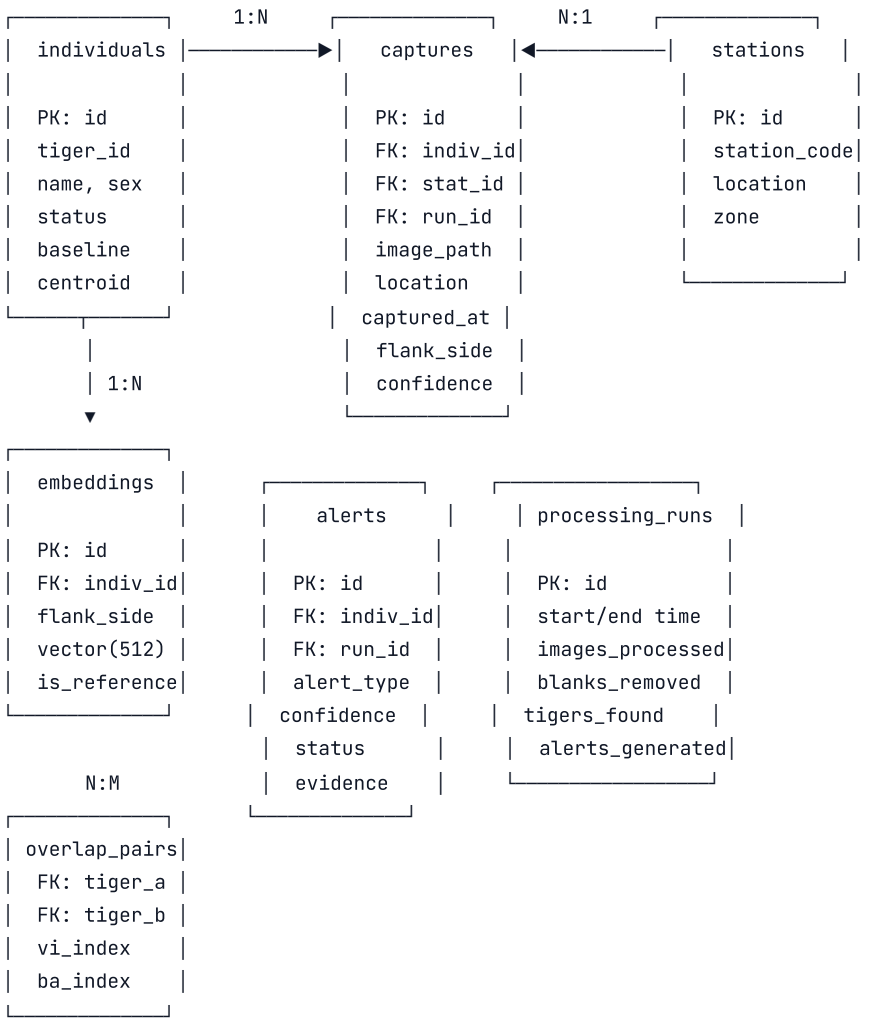
3. Technology Stack

LAYER	TECHNOLOGY	VERSION	PURPOSE
Language	Python	3.11+	Backend, ML pipeline
API Framework	FastAPI	Latest	REST API with OpenAPI 3.1 auto-docs
Database	PostgreSQL	16+	Primary relational store
Spatial	PostGIS	3.4	Geospatial queries (ST_Within, ST_DWithin, etc.)
Vector Store	pgvector	0.5+	512-d embedding similarity search (IVFFlat)
Task Queue	Celery + Redis	-	Async ML inference & batch processing
Object Storage	MinIO	Latest	S3-compatible image storage with lifecycle policies
Cache	Redis	7+	API response cache, sessions, rate limiting
ORM / Migration	SQLAlchemy + Alembic	2.0+	ORM with auto-migration management
Blank Filtering	MegaDetector V6 (YOLOv9-C)	V6	Camera trap triage via PyTorch-Wildlife
Tiger Detection	YOLOv8-L (fine-tuned)	V8	Tiger-specific object detection
Re-Identification	Siamese CNN (DenseNet-169)	-	Stripe pattern embedding extraction
Spatial Analytics	SciPy, GeoPandas, R (rpy2)	-	KDE home-range, MCP, AKDE, SECR
Containerization	Docker + Docker Compose	-	All services containerized
Monitoring	Prometheus + Grafana + Loki	-	Metrics, dashboards, log aggregation
Web Server	Nginx	-	TLS termination, reverse proxy, static assets

4. Database Architecture

4.1 Entity-Relationship Model

The database uses **PostgreSQL 16** with **PostGIS 3.4** and **pgvector 0.5+** extensions. The schema is organized around **6 core entities** with supporting tables for audit, configuration, and processing metadata.



Relationship Summary

RELATIONSHIP	CARDINALITY	DESCRIPTION
`individuals → captures`	1 : N	One individual has many captures
`stations → captures`	1 : N	One station records many captures
`individuals → embeddings`	1 : N	One individual has multiple embeddings (per flank)
`individuals → alerts`	1 : N	One individual may trigger multiple alerts

`processing_runs → captures`	1 : N	One run processes many captures
`processing_runs → alerts`	1 : N	One run may generate multiple alerts
`individuals ↔ overlap_pairs`	N : M	Many-to-many territorial overlap

4.2 Core Schema Definitions (DDL)

🐯 individuals — Tiger Identity Catalogue

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	UUID	PK DEFAULT gen_random_uuid()	Internal unique identifier
tiger_id	VARCHAR(20)	UNIQUE NOT NULL	Human-readable ID (e.g., PTR-T-001)
assigned_name	VARCHAR(100)	NULLABLE	Optional name assigned by staff
sex	VARCHAR(10)	DEFAULT 'unknown'	male / female / unknown
first_captured	TIMESTAMPTZ	NOT NULL	Timestamp of first ever capture
last_captured	TIMESTAMPTZ	NOT NULL	Timestamp of most recent capture
total_captures	INTEGER	DEFAULT 0	Running count of all captures
status	VARCHAR(20)	DEFAULT 'active'	active / absent / provisional
baseline_range	GEOMETRY(POLYGON, 4326)	NULLABLE	KDE 95% isopleth — established home range
baseline_core	GEOMETRY(POLYGON, 4326)	NULLABLE	KDE 50% isopleth — core activity area
centroid	GEOMETRY(POINT, 4326)	NULLABLE	Weighted activity centroid
range_area_sqkm	FLOAT	NULLABLE	Estimated area from KDE 95%
station_profile	JSONB	NULLABLE	Historical station visitation data
metadata	JSONB	NULLABLE	Additional flexible metadata
color_slot	INTEGER	NULLABLE	Persistent map color assignment (1-20)
created_at	TIMESTAMPTZ	DEFAULT NOW()	Record creation timestamp
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last update timestamp

📸 captures — Camera Trap Image Records

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	UUID	PK	Unique capture identifier
individual_id	UUID	FK → individuals(id)	Identified tiger (NULL if unidentified)

station_id	UUID	FK → stations(id) NOT NULL	Capture station
run_id	UUID	FK → processing_runs(id)	Processing run that ingested this
image_path	TEXT	NOT NULL	MinIO object key for original image
thumbnail_path	TEXT	NULLABLE	256×256 WebP thumbnail path
crop_path	TEXT	NULLABLE	Tiger crop image path
location	GEOMETRY(POINT, 4326)	NOT NULL	GPS coordinates (WGS 84)
captured_at	TIMESTAMPTZ	NOT NULL	Image capture timestamp (UTC)
flank_side	VARCHAR(5)	NULLABLE	L / R / unknown
match_confidence	FLOAT	NULLABLE	Cosine similarity score (0.0–1.0)
review_status	VARCHAR(20)	DEFAULT 'auto'	auto / verified / pending / rejected
reviewer_id	UUID	FK → users(id)	Human reviewer (if reviewed)
reviewed_at	TIMESTAMPTZ	NULLABLE	Review timestamp
quality_score	FLOAT	NULLABLE	Crop quality (blur, resolution, occlusion)
detection_class	VARCHAR(20)	NULLABLE	animal / person / vehicle / tiger
phash	BIGINT	NULLABLE	Perceptual hash for deduplication
metadata	JSONB	NULLABLE	EXIF data, camera model, burst info
created_at	TIMESTAMPTZ	DEFAULT NOW()	Record creation

stations — Camera Trap Station Registry

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	UUID	PK	Unique station identifier
station_code	VARCHAR(20)	UNIQUE NOT NULL	Human-readable code (e.g., ST-05)
station_name	VARCHAR(100)	NULLABLE	Descriptive name
location	GEOMETRY(POINT, 4326)	NOT NULL	GPS coordinates
zone	VARCHAR(20)	NOT NULL	core / buffer / village_adjacent
nearest_village	VARCHAR(100)	NULLABLE	Name of nearest village
village_dist_km	FLOAT	NULLABLE	Distance to nearest village (km)
deployment_status	VARCHAR(20)	DEFAULT 'active'	active / inactive / malfunction
last_maintenance	TIMESTAMPTZ	NULLABLE	Last maintenance visit date
camera_serial	VARCHAR(50)	NULLABLE	Camera device serial number
metadata	JSONB	NULLABLE	Additional station metadata

embeddings — Stripe Pattern Feature Vectors

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	UUID	PK	Unique embedding identifier
individual_id	UUID	FK → individuals(id) NOT NULL	Owning individual
flank_side	VARCHAR(5)	NOT NULL	L or R (only same-side compared)
vector	vector(512)	NOT NULL	512-d L2-normalized feature vector (pgvector)
source_image	TEXT	NOT NULL	Source flank crop image path
is_reference	BOOLEAN	DEFAULT false	Whether this is a primary reference embedding
confidence	FLOAT	NULLABLE	Quality/confidence of the embedding
model_version	VARCHAR(50)	NULLABLE	Model that generated this embedding
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation timestamp

alerts — Behavioural Deviation Alerts

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION

id	UUID	PK	Unique alert identifier
alert_type	VARCHAR(30)	NOT NULL	RANGE_SHIFT / NOVEL_STATION / BUFFER_APPROACH / PROLONGED_ABSENCE
individual_id	UUID	FK → individuals(id)	Affected tiger
run_id	UUID	FK → processing_runs(id)	Triggering processing run
confidence	VARCHAR(10)	NOT NULL	HIGH / MEDIUM / LOW
status	VARCHAR(20)	DEFAULT 'new'	new / acknowledged / resolved / false_positive
title	TEXT	NOT NULL	Human-readable alert title
description	TEXT	NULLABLE	Detailed description
evidence	JSONB	NOT NULL	Images, locations, measurements, direction
recommended_action	TEXT	NULLABLE	Suggested response action
assigned_to	UUID	FK → users(id)	Assigned officer
acknowledged_at	TIMESTAMPTZ	NULLABLE	Acknowledgment timestamp
resolved_at	TIMESTAMPTZ	NULLABLE	Resolution timestamp
notes	TEXT	NULLABLE	Officer comments
created_at	TIMESTAMPTZ	DEFAULT NOW()	Alert generation timestamp

processing_runs — Pipeline Execution Records

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	UUID	PK	Unique run identifier
status	VARCHAR(20)	DEFAULT 'ingesting'	ingesting / filtering / detecting / matching / analyzing / alerting / completed / failed
started_at	TIMESTAMPTZ	NOT NULL	Run start timestamp
completed_at	TIMESTAMPTZ	NULLABLE	Run completion timestamp
source_dir	TEXT	NULLABLE	Source directory or upload path
total_images	INTEGER	DEFAULT 0	Total images ingested
blanks_removed	INTEGER	DEFAULT 0	Blank images quarantined
tigers_detected	INTEGER	DEFAULT 0	Tiger instances detected
auto_matched	INTEGER	DEFAULT 0	Automatically matched captures
review_queued	INTEGER	DEFAULT 0	Captures queued for human review
new_enrolled	INTEGER	DEFAULT 0	New individuals enrolled
alerts_generated	INTEGER	DEFAULT 0	Alerts produced
storage_saved_mb	FLOAT	DEFAULT 0	Storage recovered via blank quarantine
operator_id	UUID	FK → users(id)	User who initiated the run
config	JSONB	NULLABLE	Run configuration (thresholds, options)
error_log	TEXT	NULLABLE	Error details if run failed
last_checkpoint	TEXT	NULLABLE	Last processed image ID for resumability

4.3 Supporting Tables

users — System User Accounts

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	UUID	PK	User identifier
username	VARCHAR(50)	UNIQUE NOT NULL	Login username
email	VARCHAR(100)	UNIQUE NOT NULL	Email address
password_hash	TEXT	NOT NULL	bcrypt hash (work factor 12)
full_name	VARCHAR(100)	NOT NULL	Display name

role	VARCHAR(20)	NOT NULL	ADMIN / BIOLOGIST / RANGE_OFFICER / FIELD_STAFF / VIEWER
is_active	BOOLEAN	DEFAULT true	Account active status
last_login	TIMESTAMPTZ	NULLABLE	Last login timestamp
created_at	TIMESTAMPTZ	DEFAULT NOW()	Account creation

🔀 overlap_pairs — Territorial Overlap Matrix

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	UUID	PK	Pair identifier
tiger_a_id	UUID	FK → individuals(id)	First individual
tiger_b_id	UUID	FK → individuals(id)	Second individual
vi_index	FLOAT	NULLABLE	Volume of Intersection (0–1)
ba_index	FLOAT	NULLABLE	Bhattacharyya's Affinity (0–1)
overlap_geom	GEOMETRY(POLYGON, 4326)	NULLABLE	Intersection polygon
run_id	UUID	FK → processing_runs(id)	Run that computed this overlap
computed_at	TIMESTAMPTZ	DEFAULT NOW()	Computation timestamp

📋 audit_log — Immutable Audit Trail (Append-Only)

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	BIGSERIAL	PK	Sequential audit ID
timestamp	TIMESTAMPTZ	DEFAULT NOW()	Event timestamp
user_id	UUID	FK → users(id)	Acting user
user_role	VARCHAR(20)		User's role at time of action
action	VARCHAR(50)	NOT NULL	Action type (e.g., REVIEW_APPROVE, ALERT_ACK)
resource_type	VARCHAR(30)		Entity type (individual, capture, alert, etc.)
resource_id	UUID		Entity identifier
before_value	JSONB		State before modification
after_value	JSONB		State after modification
ip_address	INET		Client IP address
request_hash	VARCHAR(64)		SHA-256 hash of request body

> ⚠️ **Audit Policy**

> The `audit_log` table is **append-only**. No DELETE or UPDATE operations are permitted. Retention: 2 years minimum. Archived to compressed files after 6 months.

models — AI Model Registry

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
`id`	`UUID`	PK	Model record ID
`model_name`	`VARCHAR(50)`	NOT NULL	megadetector / tiger_detector / embedding_extractor
`version`	`VARCHAR(20)`	NOT NULL	Semantic version (e.g., 1.2.0)
`weights_path`	`TEXT`	NOT NULL	MinIO path to model weights
`architecture`	`VARCHAR(50)`		e.g., YOLOv9-C, DenseNet-169
`is_active`	`BOOLEAN`	DEFAULT false	Currently active version
`metrics`	`JSONB`		Validation metrics (mAP, accuracy, etc.)
`training_config`	`JSONB`		Hyperparameters, dataset info
`uploaded_at`	`TIMESTAMPTZ`	DEFAULT NOW()	Upload timestamp

system_config — Runtime Configuration

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
`key`	`VARCHAR(100)`	PK	Configuration key
`value`	`JSONB`	NOT NULL	Configuration value
`description`	`TEXT`		Human-readable description
`updated_by`	`UUID`	FK → users(id)	Last updater
`updated_at`	`TIMESTAMPTZ`	DEFAULT NOW()	Last update time

4.4 Indexing Strategy

INDEX TYPE	TABLE.COLUMN(S)	PURPOSE
B-TREE	captures.individual_id	Fast joins to individuals
B-TREE	captures.station_id	Fast joins to stations
B-TREE	captures.run_id	Fast joins to processing_runs
COMPOSITE	captures(individual_id, captured_at DESC)	Efficient capture history queries
PARTIAL	captures WHERE review_status='pending'	Fast review queue retrieval
BRIN	captures.created_at	Time-range partition pruning on large tables
GIST	captures.location	Spatial queries (ST_Within, ST_DWithin)
GIST	stations.location	Station proximity searches
GIST	individuals.baseline_range	Spatial intersection tests
GIST	individuals.centroid	Centroid proximity queries
IVFFLAT	embeddings.vector (vector_cosine_ops)	Approximate nearest neighbor search (nlist=100)
B-TREE	alerts.alert_type	Filter by alert type
B-TREE	alerts.status	Filter by alert status
B-TREE	alerts.individual_id	All alerts for a tiger
B-TREE	individuals.tiger_id	Unique index on human-readable ID
B-TREE	individuals.status	Filter by status (active/absent/provisional)

4.5 Vector Store Configuration

PARAMETER	VALUE	NOTES
Extension	pgvector 0.5+	Compiled with PostgreSQL 16
Vector Dimension	512	Matching Siamese CNN output
Distance Metric	Cosine Distance	⟺ operator (1 - cosine_similarity)
Index Type	IVFFlat	For catalogues >500 embeddings; exact scan below
nlist	ceil(sqrt(N))	N = total embeddings
nprobe	max(10, nlist/10)	Search accuracy vs speed tradeoff

```
-- Example: Vector similarity search (same-flank matching)
SELECT individual_id,
       1 - (vector ⟺ $query_vector) AS similarity
```

```
FROM embeddings
WHERE flank_side = 'L'
ORDER BY vector ⟺ $query_vector
LIMIT 10;
```

5. API Architecture & Endpoints

5.1 REST API Design Principles

- All endpoints follow RESTful conventions with resource-oriented URLs
- Request/response bodies: **JSON** (Content-Type: application/json). File uploads: multipart/form-data
- URL versioning: `/api/v1/...`
- Pagination: **cursor-based** — `?cursor=xxx&per_page=20`
- Auto-generated OpenAPI 3.1 spec at `/api/docs` (Swagger UI) and `/api/redoc`
- Standard HTTP status codes: 200, 201, 202, 400, 401, 403, 404, 422, 429, 500

5.2 Authentication & RBAC

ROLE	DASHBOARD	CATALOGUE	UPLOAD	REVIEW	ALERTS	MAP/EXPORT	SETTINGS	GPS FULL
ADMIN	✅	R/W	✅	✅	R/W	✅	✅	✅
BIOLOGIST	✅	R/W	✅	✅	R/W	✅	Partial	✅
RANGE_OFFICER	✅	Read	✅	—	R/W	✅ (GPS rounded)	—	—
FIELD_STAFF	Limited	—	✅	—	View	—	—	—
VIEWER	✅	Read	—	—	View	View only	—	—

> 🔐 **Authentication Flow**
>
> JWT bearer tokens via `POST /api/v1/auth/login`. Access tokens expire in **1 hour**; refresh tokens in **7 days**. Password storage: **bcrypt** (work factor 12, min 12 chars). Service-to-service: API keys (256-bit random).

5.3 Complete Endpoint Catalogue

AUTHENTICATION (3 ENDPOINTS)

METHOD	ENDPOINT	DESCRIPTION	AUTH
POST	`/api/v1/auth/login`	Authenticate user, return JWT tokens	Public
POST	`/api/v1/auth/refresh`	Refresh access token	Refresh Token
POST	`/api/v1/auth/logout`	Invalidate tokens	Any

USER MANAGEMENT (5 ENDPOINTS)

METHOD	ENDPOINT	DESCRIPTION	AUTH
GET	/api/v1/users	List all users	Admin
POST	/api/v1/users	Create new user	Admin
GET	/api/v1/users/{id}	Get user profile	Admin
PUT	/api/v1/users/{id}	Update user (role, status)	Admin
DELETE	/api/v1/users/{id}	Disable/delete user	Admin

BLANK IMAGE FILTERING — MODULE 1 (6 ENDPOINTS)

METHOD	ENDPOINT	DESCRIPTION	AUTH
POST	/api/v1/filtering/run	Start filtering run {source_dir, threshold, dry_run}	Admin, Bio
GET	/api/v1/filtering/runs/{run_id}	Get run status & summary	Any
GET	/api/v1/filtering/runs/{run_id}/report	Full report with per-station breakdown	Any
POST	/api/v1/quarantine/{image_id}/restore	Restore quarantined image	Admin, Bio
DELETE	/api/v1/quarantine/purge	Permanently delete old quarantined images	Admin
GET	/api/v1/filtering/config	Get/update filter configuration	Admin

TIGER IDENTIFICATION — MODULE 2 (8 ENDPOINTS)

METHOD	ENDPOINT	DESCRIPTION	AUTH
POST	/api/v1/identification/run	Start identification pipeline	Admin, Bio
GET	/api/v1/identification/runs/{task_id}	Get identification run results	Any
GET	/api/v1/catalogue	List individuals (paginated, filterable)	Any
GET	/api/v1/catalogue/{individual_id}	Full individual profile + history	Any
GET	/api/v1/catalogue/{individual_id}/images	Capture images (filterable by flank, station, date)	Any
GET	/api/v1/review/queue	Get pending ambiguous matches	Admin, Bio
PUT	/api/v1/review/{review_id}	Confirm/reject/merge match	Admin, Bio
POST	/api/v1/catalogue/{id}/merge	Merge duplicate individuals	Admin, Bio

SPATIAL OCCUPANCY — MODULE 3 (6 ENDPOINTS)

METHOD	ENDPOINT	DESCRIPTION	AUTH
POST	/api/v1/occupancy/compute	Trigger KDE/MCP computation	Admin, Bio
GET	/api/v1/occupancy/individuals/{id}	Individual range stats + GeoJSON	Any
GET	/api/v1/occupancy/overlap	Pairwise overlap matrix + polygons	Any
GET	/api/v1/occupancy/map/layers	All spatial layers as GeoJSON	Any
GET	/api/v1/occupancy/export	Export as Shapefile/GeoJSON/KML/PDF	Bio+
GET	/api/v1/occupancy/summary	Reserve-wide summary statistics	Any

ALERTS — MODULE 4 (7 ENDPOINTS)

METHOD	ENDPOINT	DESCRIPTION	AUTH
POST	/api/v1/alerts/generate	Trigger alert generation from run	Auto/Admin
GET	/api/v1/alerts	List alerts (filterable, paginated)	Any
GET	/api/v1/alerts/{alert_id}	Full alert detail with evidence	Any
PUT	/api/v1/alerts/{alert_id}	Update status (acknowledge/resolve/false_positive)	Officer+
GET	/api/v1/alerts/trends	Time-series alert analytics	Bio+
POST	/api/v1/alerts/config	Update alert rule thresholds	Admin
GET	/api/v1/alerts/digest	PDF report of alerts by period	Any

STATIONS (5 ENDPOINTS)

METHOD	ENDPOINT	DESCRIPTION	AUTH
GET	/api/v1/stations	List all stations	Any
POST	/api/v1/stations	Create station / bulk CSV import	Admin
GET	/api/v1/stations/{id}	Station detail + captures	Any
PUT	/api/v1/stations/{id}	Update station metadata	Admin, Bio
DELETE	/api/v1/stations/{id}	Deactivate station	Admin

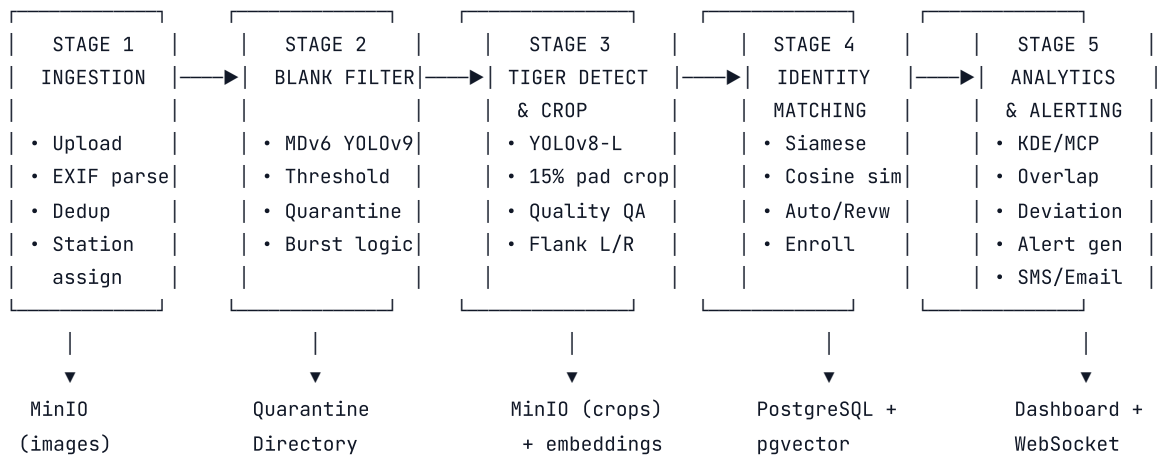
PROCESSING RUNS & SYSTEM HEALTH (5 ENDPOINTS)

METHOD	ENDPOINT	DESCRIPTION	AUTH
GET	/api/v1/runs	List all processing runs	Any
GET	/api/v1/runs/{id}	Run detail with full metrics	Any

GET	/api/v1/runs/{id}/compare	Compare two runs side-by-side	Bio+
GET	/api/v1/health	Liveness check	Public
GET	/api/v1/ready	Readiness (DB + GPU check)	Public

6. Processing Pipeline Architecture

The system processes camera trap data through a **5-stage sequential pipeline** managed by Celery task queues with Redis broker. Each stage is independently recoverable via checkpointing.



Stage Details

STAGE	MODEL / METHOD	INPUT	OUTPUT	PERFORMANCE TARGET
1. Ingestion	Pillow + exifread + pHash	Raw image directories	Indexed images in MinIO + metadata in DB	—
2. Blank Filter	MegaDetector V6 (YOLOv9-C, 1280×1280)	All ingested images	BLANK / SUBJECT / BOUNDARY classification	≥1,000 img/min (GPU)
3. Tiger Detection	Fine-tuned YOLOv8-L (min conf: 0.4)	Retained (non-blank) images	Tiger crops (448×448) with flank annotation	≥500 img/min (GPU)
4. Identity Matching	Siamese CNN (DenseNet-169) → 512-d → cosine sim	Tiger flank crops	Matches (>0.85 auto, 0.60-0.85 review, <0.60 new)	<5s per image
5. Analytics + Alerts	SciPy KDE + R adehabitatHR + statistical rules	Updated individual records	KDE polygons, overlap matrix, classified alerts	<30min for 80 tigers

Matching Thresholds

SIMILARITY RANGE	ACTION	DESCRIPTION
`≥ 0.85`	**AUTO_MATCH**	Automatic assignment. Reference embedding updated (10% weight).
`0.60 – 0.85`	**REVIEW_QUEUE**	Surfaced to human reviewer with side-by-side comparison UI.
`< 0.60`	**NEW_INDIVIDUAL**	New enrollment triggered. ID: PTR-T-{NNN}. Status: provisional.

Alert Rule Engine

ALERT TYPE	TRIGGER CONDITION	DEFAULT THRESHOLD	PRIORITY
RANGE_SHIFT	Centroid displacement exceeds threshold	Core: 15 sq km shift. Buffer: 5 km linear.	Medium–High
NOVEL_STATION	Capture at station not in historical profile	>5 km from range: HIGH. 2-5 km: MED. <2 km: LOW.	Variable
BUFFER_APPROACH	Capture at buffer/village-adjacent station not in profile	Always triggered for new buffer captures.	**Always HIGH**
PROLONGED_ABSENCE	Regular individual not detected in current run	1st miss: LOW. 2nd: MEDIUM. 3rd+: HIGH.	Escalating

7. Data Ingestion & ETL Pipeline

Ingestion Modes

- **Directory Scan:** Recursive scan of mounted filesystem path (SD card contents). Supports JPEG, PNG, TIFF, BMP.
- **Web Upload:** ZIP archive upload via dashboard (max 10 GB per upload).

Metadata Extraction

SOURCE	FIELDS EXTRACTED	FALLBACK
EXIF	DateTimeOriginal, Make/Model, GPS, Orientation	File modification time (flagged as 'estimated')
Station Assignment	Nearest station within 100m of EXIF GPS	Directory name regex pattern or CSV mapping
Camera Health	Serial number, operational days	Manual entry

Deduplication

- **pHash (64-bit):** Computed for every image. Hamming distance = 0 → exact duplicate (skipped). Hamming ≤ 5 → near-duplicate (flagged but ingested).
- **Cross-run:** New images checked against last 3 runs for duplicates.

Data Standards

- Image metadata: **Camtrap DP** (Camera Trap Data Package) for interoperability with Wildlife Insights & LILA BC.
- Spatial data: **WGS 84 (EPSG:4326)**. All timestamps stored in **UTC** with IST conversion for display.
- Export: COCO Camera Traps annotation format for ML community compatibility.

8. Security Architecture

LAYER	MECHANISM	STANDARD
Data at Rest	PostgreSQL TDE + MinIO server-side encryption	AES-256-GCM
Data in Transit	Nginx TLS termination + internal PostgreSQL SSL	TLS 1.3 / TLS 1.2+
Backups	GPG encryption before external storage	RSA-4096
GPS Coordinates	Full precision: ADMIN+BIO only. Rounded (3 decimals) for RANGE_OFFICER+	~111m precision
Authentication	JWT bearer tokens (1hr access, 7d refresh)	bcrypt (WF=12)
Authorization	5-role RBAC with endpoint-level permission checks	Custom middleware
Audit	Append-only audit_log table, 2-year retention	Full request tracing

Rate Limiting	Redis sliding window — 100 req/min auth, 20 unauth, 5 for heavy ops	429 + Retry-After
Account Lockout	5 failures in 15 min → 30-min lockout	Brute-force prevention
Network	All services on Docker bridge. Only Nginx ports (80/443) exposed.	Zero-trust internal

9. Infrastructure & Deployment

Deployment Topology

Primary deployment: **Single-server on-premise** at PTR headquarters, running a Docker Compose stack.

SERVICE	PORT (INTERNAL)	PORT (EXPOSED)	RESOURCES
Nginx	—	80 (HTTP), 443 (HTTPS)	TLS termination, reverse proxy
FastAPI	8000	—	API + Celery workers
PostgreSQL 16	5432	Optional (SSH tunnel)	16+ GB RAM, 16 GB shared_buffers
Redis 7	6379	—	Cache + Celery broker
MinIO	9000/9001	—	Image storage, erasure coding
Prometheus	9090	—	Metrics collection
Grafana	3000	Via Nginx at /monitoring	Dashboards + alerting

Hardware Requirements

COMPONENT	MINIMUM SPECIFICATION
Processing Server	32+ GB RAM, NVIDIA RTX 3090/4090 (16+ GB VRAM), 2+ TB NVMe SSD
Database	16+ GB RAM, 1+ TB storage (can co-locate)
Edge (Future)	NVIDIA Jetson Orin Nano 8GB or Google Coral Dev Board
Networking	Gigabit Ethernet (local) + 4G/satellite uplink (alerts)

10. Performance & Caching Strategy

COMPONENT	TARGET (GPU)	TARGET (CPU FALLBACK)
Blank Filtering	≥1,000 img/min, <60ms/img p95	≥100 img/min
Tiger Detection	≥500 img/min, <120ms/img p95	—
Embedding Extraction	≥200 crops/min, <300ms/crop	<500ms/crop
Vector Search (top-10)	<50ms for 10,000 embeddings	
KDE (per individual)	<30s for 100 capture points	
Full Reserve Occupancy	<30min for 80 individuals	
API Response (read)	p95 < 200ms	

Dashboard Load	<3s initial, <5s with map layers

Caching Layers

- **Redis:** API response cache (5min TTL for dashboard, 1hr for catalogue), sessions, rate limits, embedding cache.
- **MinIO Thumbnails:** 256×256 WebP generated on first access, served directly on repeat.
- **Spatial Cache:** KDE GeoJSON polygons cached per individual, invalidated on new data ingestion.

GPU Optimization

- **Mixed Precision (FP16):** ~40% VRAM reduction, <1% accuracy impact.
- **Dynamic Batching:** Queue-based dispatch at optimal batch size per VRAM availability.
- **Model Warmup:** 5 dummy images on startup to initialize CUDA kernels.
- **OOM Protection:** Auto batch-size reduction at >90% VRAM usage.

11. Monitoring & Observability

CATEGORY	METRICS	TOOL
Application	Request count/latency (per endpoint), Celery queue depth, processing progress	Prometheus
ML Models	Inference latency per model, GPU utilization, VRAM usage, blank ratio, accuracy drift	Prometheus
Infrastructure	CPU/memory/disk, PostgreSQL connections, Redis hit rate, MinIO storage	Prometheus
Business	Catalogue size, new enrollments/month, alert count by type, review backlog	Grafana
Logging	Structured JSON logs with trace_id, 90d retention in Loki, 1y in archive	Loki
System Alerts	GPU OOM, disk >85%, PG connection exhaust, worker crash, API error >5%	Grafana Alerting

12. Backup & Disaster Recovery

COMPONENT	METHOD	FREQUENCY	RETENTION	RPO
Database	pg_dump + WAL archiving (zstd + GPG)	Daily (2 AM IST)	30 days (daily), 1 year (monthly)	24 hours
Images (MinIO)	mc mirror to external NAS/USB	Weekly + post-major run	90 days	7 days
Configuration	Git-versioned Docker/env/certs	On every change	Indefinite	0 hours
Model Weights	MinIO bucket /models/	On upload	All versions	0 hours

Recovery Targets

- **RTO (same hardware):** < 4 hours
- **RTO (replacement hardware):** < 8 hours (including OS + Docker setup)
- **Monthly DR Drill:** Restore from backup on isolated VM, run integration test suite.

> ✅ **Data Integrity**
>
> All database backups verified with `pg_restore --list` and row-count comparison. MinIO objects verified with MD5 checksums (1% sampled per run). All quarantined images and database records are never permanently deleted without explicit admin action + dual confirmation.